Task Documentation: JavaScript Mini Project (Calculator App)

Student Name: Gokul V

Date: October 6, 2026

1) Project Goal:

The objective of this task is to build an interactive, responsive Scientific Calculator Application using core Web Technologies (HTML5, Tailwind CSS, and plain JavaScript). The app highlights advanced DOM manipulation, event handling, string parsing, and trigonometric/logarithmic mathematical evaluations using the JavaScript Math library.

2) Key Features:

➔ **Basic Arithmetic Operations:**
Addition, Subtraction, Multiplication, Division, and Percentage calculations.

➔ **Scientific Functions:**
Trigonometric ratios ($\sin, \cos, \tan$ calculated in degrees), square root ($\sqrt{x}$), powers (x^2, x^y), logarithms ($\log_{10}$, $\ln$), and constants (π, e).

➔ **Interactive UI & Dual Display:**
Top bar displays calculation history while the main screen renders live inputs and outputs.

➔ **Keyboard Support:**
Supports standard physical keyboard inputs for digits, basic arithmetic operators, backspace, and clear commands.

➔ **Error Handling:**
Protects against illegal mathematical operations such as division by zero, square roots of negative numbers, and non-positive logarithms.

3) Project File Structure:

```
scientific-calculator/
├── index.html   # Calculator DOM structure, display screen, and button layout grid
├── styles.css   # Custom button styles, animations, and hover state styling
└── script.js    # Scientific calculation algorithms, DOM listeners, and state management
```

4) Code Implementation:

File 1: index.html

```
<!DOCTYPE html>

<html lang="en">

<head>

    <meta charset="UTF-8">

    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <title>JS Mini Project - Scientific Calculator</title>
```

```
    <!-- Tailwind CSS CDN -->
    <script src="https://cdn.tailwindcss.com"></script>
    <!-- Custom CSS -->
    <link rel="stylesheet" href="styles.css">
</head>
<body class="bg-slate-950 text-slate-100 min-h-screen flex items-center justify-center p-4">
    <div class="w-full max-w-lg bg-slate-900 border border-slate-800 rounded-3xl shadow-2xl p-6">
        <!-- Header -->
        <div class="flex items-center justify-between pb-4 mb-4 border-b border-slate-800">
            <h1 class="text-lg font-bold text-indigo-400 flex items-center gap-2">
                <span> 🧪 </span> Scientific Calculator
            </h1>
            <span class="text-xs font-semibold px-2.5 py-1 bg-indigo-950 text-indigo-300 border border-indigo-800 rounded-full">
                DOM Project
            </span>
        </div>
        <!-- Calculator Display Screen -->
        <div class="bg-slate-950 border border-slate-800 rounded-2xl p-4 mb-5 text-right shadow-inner">
            <div id="history-display" class="text-xs text-slate-500 h-5 overflow-hidden font-mono truncate"></div>
            <div id="main-display" class="text-3xl font-bold text-white h-10 overflow-hidden font-mono truncate">0</div>
        </div>
        <!-- Keypad Grid (5 columns for Scientific Layout) -->
        <div class="grid grid-cols-5 gap-2.5">
            <!-- Row 1: Scientific Functions -->
            <button class="btn btn-sci" data-func="sin">sin</button>
            <button class="btn btn-sci" data-func="cos">cos</button>
            <button class="btn btn-sci" data-func="tan">tan</button>
            <button class="btn btn-action" data-action="clear">AC</button>
```

```
<button class="btn btn-action" data-action="delete">DEL</button>
<!-- Row 2: Scientific Functions -->
<button class="btn btn-sci" data-func="pow">x<sup>y</sup></button>
<button class="btn btn-sci" data-func="square">x²</button>
<button class="btn btn-sci" data-func="sqrt">√</button>
<button class="btn btn-action" data-action="percent">%</button>
<button class="btn btn-operator" data-operator="÷">÷</button>
<!-- Row 3 -->
<button class="btn btn-sci" data-func="log">log</button>
<button class="btn btn-number" data-number="7">7</button>
<button class="btn btn-number" data-number="8">8</button>
<button class="btn btn-number" data-number="9">9</button>
<button class="btn btn-operator" data-operator="×">×</button>
<!-- Row 4 -->
<button class="btn btn-sci" data-func="ln">ln</button>
<button class="btn btn-number" data-number="4">4</button>
<button class="btn btn-number" data-number="5">5</button>
<button class="btn btn-number" data-number="6">6</button>
<button class="btn btn-operator" data-operator="-">-</button>

<!-- Row 5 -->
<button class="btn btn-sci" data-func="pi">π</button>
<button class="btn btn-number" data-number="1">1</button>
<button class="btn btn-number" data-number="2">2</button>
<button class="btn btn-number" data-number="3">3</button>
<button class="btn btn-operator" data-operator="+">+</button>
<!-- Row 6 -->
<button class="btn btn-sci" data-func="e">e</button>
<button class="btn btn-number" data-number="0">0</button>
```

```
        <button class="btn btn-number" data-number=".">.</button>
        <button class="btn btn-equals col-span-2" data-action="equals">=</button>
      </div>
    </div>
    <!-- Script loaded with defer -->
    <script src="script.js" defer></script>
</body>
</html>
```

File 2: styles.css

```
@import url('https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600;700&display=swap');
body {
    font-family: 'Inter', sans-serif;
}
/* Button Base Styles */
.btn {
    font-size: 0.95rem;
    font-weight: 600;
    padding: 0.75rem 0;
    border-radius: 0.75rem;
    transition: all 0.15s ease-in-out;
    user-select: none;
    cursor: pointer;
    display: flex;
    align-items: center;
    justify-content: center;
}
.btn:active {
    transform: scale(0.94);
```

```
}
/* Scientific Function Button */
.btn-sci {
    background-color: #0284c7;
    border: 1px solid #0369a1;
    color: #e0f2fe;
}
.btn-sci:hover {
    background-color: #0369a1;
    color: #ffffff;
}
/* Number Button */
.btn-number {
    background-color: #0f172a;
    border: 1px solid #1e293b;
    color: #f1f5f9;
}
.btn-number:hover {
    background-color: #1e293b;
    border-color: #334155;
}
/* Action Button (AC, DEL, %) */
.btn-action {
    background-color: #1e1b4b;
    border: 1px solid #312e81;
    color: #a5b4fc;
}
.btn-action:hover {
    background-color: #312e81;
```

```
    color: #ffffff;
}
/* Operator Button (+, -, ×, ÷) */
.btn-operator {
    background-color: #312e81;
    border: 1px solid #4338ca;
    color: #c7d2fe;
}
.btn-operator:hover {
    background-color: #4338ca;
    color: #ffffff;
}
/* Equals Button */
.btn-equals {
    background-color: #4f46e5;
    border: 1px solid #6366f1;
    color: #ffffff;
    font-size: 1.125rem;
}
.btn-equals:hover {
    background-color: #6366f1;
}
```

File 3: script.js

```
document.addEventListener("DOMContentLoaded", () => {
    // DOM Elements
    const mainDisplay = document.getElementById("main-display");
    const historyDisplay = document.getElementById("history-display");
    // State Variables
```

```
let currentInput = "0";
let previousInput = "";
let operator = null;
let shouldResetDisplay = false;
// Update Display UI
function updateDisplay() {
    mainDisplay.textContent = currentInput;
    if (operator !== null) {
        historyDisplay.textContent = `${previousInput} ${operator}`;
    } else {
        historyDisplay.textContent = "";
    }
}
// Append Number / Decimal
function appendNumber(number) {
    if (currentInput === "0" || shouldResetDisplay) {
        currentInput = number === "." ? "0." : number;
        shouldResetDisplay = false;
    } else {
        if (number === "." && currentInput.includes(".")) return;
        currentInput += number;
    }
    updateDisplay();
}
// Choose Operator
function chooseOperator(op) {
    if (operator !== null && !shouldResetDisplay) {
        calculate();
    }
```

```
    previousInput = currentInput;
    operator = op;
    shouldResetDisplay = true;
    updateDisplay();
}
// Execute Scientific Functions
function handleScientific(func) {
    let num = parseFloat(currentInput);
    let result;
    switch (func) {
        case "sin":
            result = Math.sin(num * (Math.PI / 180)); // Degrees
            historyDisplay.textContent = `sin(${currentInput}°)`;
            break;
        case "cos":
            result = Math.cos(num * (Math.PI / 180)); // Degrees
            historyDisplay.textContent = `cos(${currentInput}°)`;
            break;
        case "tan":
            result = Math.tan(num * (Math.PI / 180)); // Degrees
            historyDisplay.textContent = `tan(${currentInput}°)`;
            break;
        case "sqrt":
            if (num < 0) {
                alert("Invalid Input for Square Root");
                return;
            }
            result = Math.sqrt(num);
            historyDisplay.textContent = `√(${currentInput})`;
```

```
        break;
    case "square":
        result = Math.pow(num, 2);
        historyDisplay.textContent = `(${currentInput})²`;
        break;
    case "log":
        if (num <= 0) {
            alert("Invalid Input for Logarithm");
            return;
        }
        result = Math.log10(num);
        historyDisplay.textContent = `log(${currentInput})`;
        break;
    case "ln":
        if (num <= 0) {
            alert("Invalid Input for Natural Log");
            return;
        }
        result = Math.log(num);
        historyDisplay.textContent = `ln(${currentInput})`;
        break;
    case "pow":
        chooseOperator("^");
        return;
    case "pi":
        result = Math.PI;
        historyDisplay.textContent = "π";
        break;
    case "e":
```

```
            result = Math.E;
            historyDisplay.textContent = "e";
            break;
        default:
            return;
    }
    result = Math.round(result * 100000000) / 100000000;
    currentInput = result.toString();
    shouldResetDisplay = true;
    mainDisplay.textContent = currentInput;
}
// Standard Calculation
function calculate() {
    if (operator === null || shouldResetDisplay) return;
    let result;
    const prev = parseFloat(previousInput);
    const current = parseFloat(currentInput);
    if (isNaN(prev) || isNaN(current)) return;
    switch (operator) {
        case "+":
            result = prev + current;
            break;
        case "-":
            result = prev - current;
            break;
        case "×":
            result = prev * current;
            break;
        case "/":
```

```
            if (current === 0) {
                alert("Cannot divide by zero!");
                clearAll();
                return;
            }
            result = prev / current;
            break;
        case "^":
            result = Math.pow(prev, current);
            break;
        default:
            return;
    }
    result = Math.round(result * 100000000) / 100000000;
    historyDisplay.textContent = `${previousInput} ${operator} ${currentInput} =`;
    currentInput = result.toString();
    operator = null;
    shouldResetDisplay = true;
    mainDisplay.textContent = currentInput;
}
// Clear All
function clearAll() {
    currentInput = "0";
    previousInput = "";
    operator = null;
    shouldResetDisplay = false;
    updateDisplay();
}
// Delete Last Digit
```

```
function deleteDigit() {
    if (shouldResetDisplay) return;
    if (currentInput.length === 1 || (currentInput.length === 2 && currentInput.startsWith("-"))) {
        currentInput = "0";
    } else {
        currentInput = currentInput.slice(0, -1);
    }
    updateDisplay();
}
// Percentage
function percentage() {
    currentInput = (parseFloat(currentInput) / 100).toString();
    updateDisplay();
}
// Event Delegation for Button Clicks
document.querySelector(".grid").addEventListener("click", (e) => {
    const target = e.target.closest("button");
    if (!target) return;
    const number = target.dataset.number;
    const op = target.dataset.operator;
    const func = target.dataset.func;
    const action = target.dataset.action;
    if (number !== undefined) appendNumber(number);
    if (op !== undefined) chooseOperator(op);
    if (func !== undefined) handleScientific(func);
    if (action === "clear") clearAll();
    if (action === "delete") deleteDigit();
    if (action === "percent") percentage();
    if (action === "equals") calculate();
```

```
  });
  // Keyboard Shortcuts
  window.addEventListener("keydown", (e) => {
    if ((e.key >= "0" && e.key <= "9") || e.key === ".") appendNumber(e.key);
    if (e.key === "+") chooseOperator("+");
    if (e.key === "-") chooseOperator("-");
    if (e.key === "*") chooseOperator("×");
    if (e.key === "/") chooseOperator("÷");
    if (e.key === "^") chooseOperator("^");
    if (e.key === "Enter" || e.key === "=") {
      e.preventDefault();
      calculate();
    }
    if (e.key === "Backspace") deleteDigit();
    if (e.key === "Escape") clearAll();
  });
});
```

5) Deployment Steps (GitHub Pages):

Repository Configuration: Created a public repository on GitHub.

File Upload: Pushed index.html and styles.css to the main branch root directory.

Pages Settings:

Navigated to Settings > Pages.

Selected Deploy from a branch.

Set Branch to main / / (root).

Site Verification: Confirmed successful deployment via the live GitHub Pages domain.

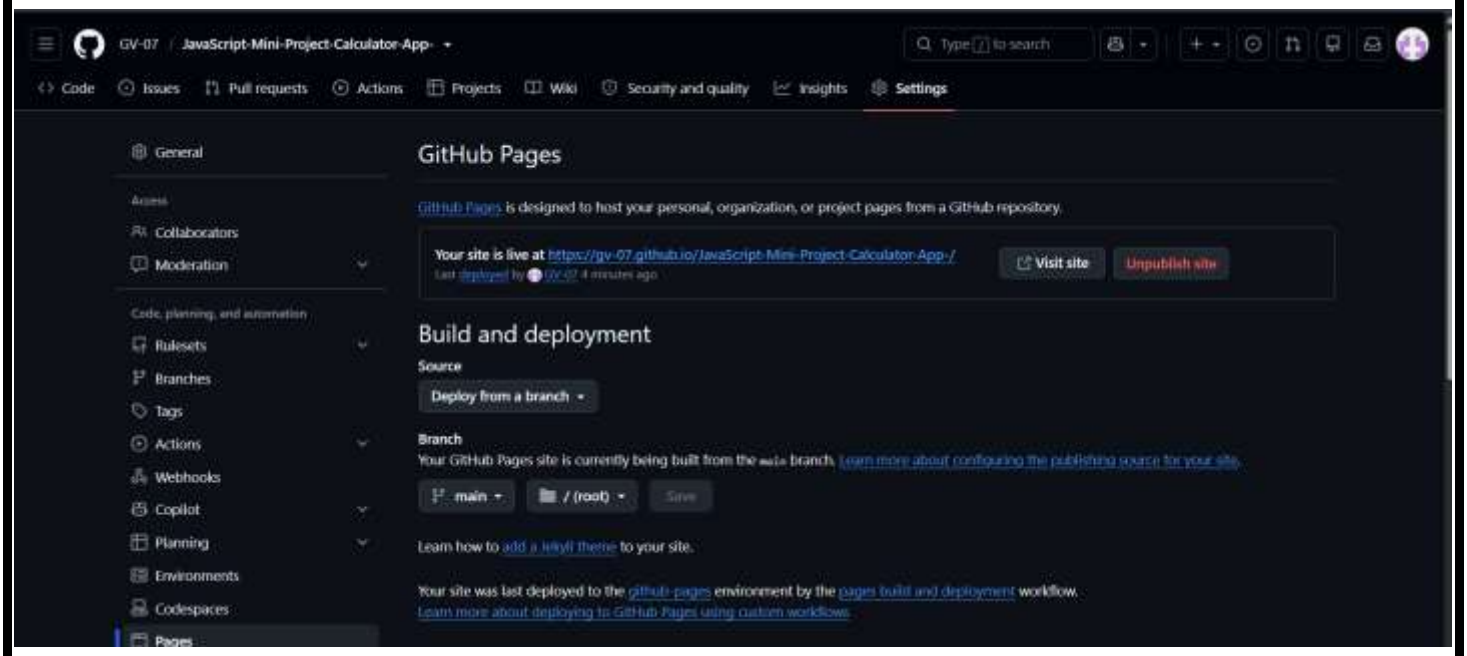
6) Project Links:

GitHub Repository Link: https://github.com/GV-07/JavaScript-Mini-Project-Calculator-App-

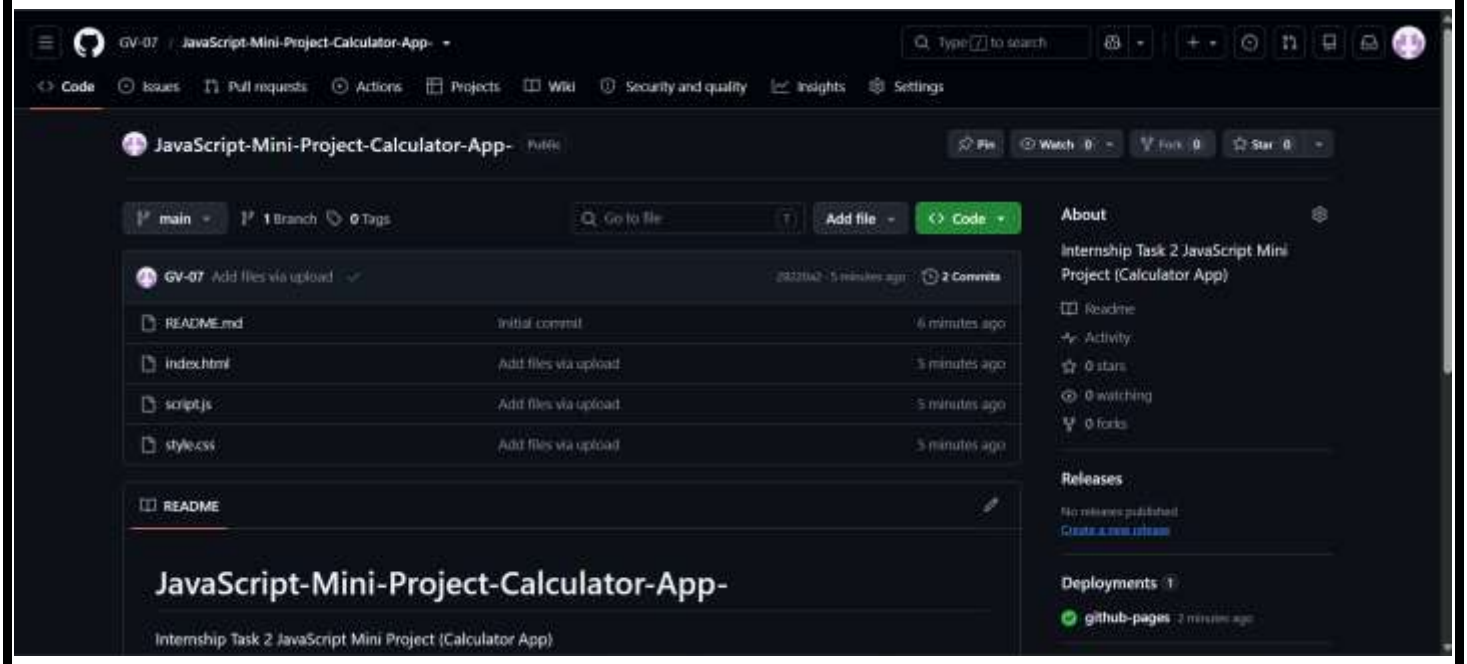
Hosted Live Website Link: https://gv-07.github.io/JavaScript-Mini-Project-Calculator-App-/

7) Visual Screenshots:

GitHub Repository Code Upload:



GitHub Pages Live Deployment Settings:



Live Portfolio Preview:

